

INF01151 - Sistemas Operacionais II

Trabalho Prático - Parte 2

Henry R. Piceni¹, João P. F. Pereira¹, Pedro H. F. Fleck¹, Luís F. L. França¹

¹Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

1. Introdução

Este relatório detalha a implementação de um sistema distribuído com tolerância a falhas, desenvolvido para a disciplina *INF01151 - Sistemas Operacionais II*. O foco do trabalho é a aplicação de uma estratégia de replicação passiva [Schneider 1990], onde um servidor primário processa as operações e as propaga de forma síncrona aos backups. Nas seções seguintes, serão discutidos em detalhe como a replicação passiva foi implementada. Adicionalmente, o documento abordará o ambiente de testes utilizado e os problemas encontrados durante o desenvolvimento.

2. Primeira Parte do Trabalho

A primeira parte do trabalho consistiu na implementação de um serviço de sincronização de arquivos semelhante ao *Dropbox*, utilizando *threads*, *sockets* TCP e mecanismos de sincronização como *mutex* e semáforos. O sistema foi desenvolvido em C++ para ambientes *Unix* e incluiu um servidor capaz de gerenciar múltiplos usuários e dispositivos simultaneamente, garantindo a consistência dos dados através de sincronização automática do diretório `sync_dir`. O servidor utiliza *threads* para atender clientes concorrentemente, enquanto o cliente monitora alterações locais e as propaga para o servidor e outros dispositivos. Foram implementados comandos básicos como `upload`, `download`, `delete` e listagem de arquivos, além de um mecanismo para evitar loops infinitos durante a sincronização.

3. Ambiente de Testes

Os testes foram realizados em um ambiente isolado utilizando contêiner Docker baseado na imagem oficial do Ubuntu 22.04. O contêiner foi executado em um MacBook equipado com chip Apple M2 e 8 GB de memória RAM, rodando o sistema operacional macOS Sequoia (15.3.1). Dentro do contêiner, foram instaladas as ferramentas essenciais de compilação, incluindo o g++ (versão 11.4.0), make (versão 4.3), e cmake (versão 3.22.1), além de dependências como `libpthread-stubs0-dev`, `git`, `vim` e `net-tools`.

O código-fonte do projeto foi copiado para o diretório de trabalho `/app` dentro do contêiner, conforme especificado no `Dockerfile`. A compilação foi realizada a partir desse diretório com os comandos `make clean & make`. É importante observar que o contêiner utiliza o kernel fornecido pelo sistema operacional hospedeiro (macOS), mas todo o ambiente de compilação e execução está restrito ao sistema Ubuntu dentro do Docker, garantindo portabilidade e reprodutibilidade nos testes.

O ambiente de testes foi orquestrado por meio do Docker Compose. Foram definidos dois serviços principais: um servidor e um cliente, ambos construídos a partir

do mesmo contexto Docker localizado no diretório do projeto. O serviço de servidor é responsável por iniciar a aplicação principal, escutando em uma porta específica que é exposta e mapeada diretamente para o host, permitindo o acesso externo ao sistema em execução. O projeto do host é montado como volume compartilhado dentro dos contêineres, o que facilita o desenvolvimento iterativo e garante que alterações no código estejam imediatamente disponíveis no ambiente de testes.

O serviço de cliente é configurado para ser executado em modo interativo, com terminal alocado e entrada padrão aberta, permitindo a execução manual de comandos dentro do contêiner. Essa configuração é particularmente útil para depuração e testes pontuais. Ambos os serviços estão conectados a uma rede virtual interna, gerenciada pelo Docker com driver do tipo bridge, permitindo comunicação direta entre os contêineres por nome de serviço, o que simula de forma realista um ambiente cliente-servidor.

Essa infraestrutura proporciona um ambiente flexível e reproduzível para o desenvolvimento e testes de aplicações distribuídas, combinando automação na execução com a possibilidade de intervenção manual quando necessário.

4. Desenvolvimento

Conforme a figura 1, retirada da especificação do trabalho, a arquitetura do sistema segue o modelo de replicação passiva, com um servidor primário responsável por processar operações e propagar atualizações de forma síncrona para os servidores de backup. Os clientes interagem com o sistema por meio de um *frontend* que encaminha as requisições ao primário. Em caso de falha do primário, o sistema utilizaria o algoritmo *Bully* [Garcia-Molina 1982] para eleger um novo líder entre os servidores disponíveis, porém isto não foi implementado neste caso de estudo.

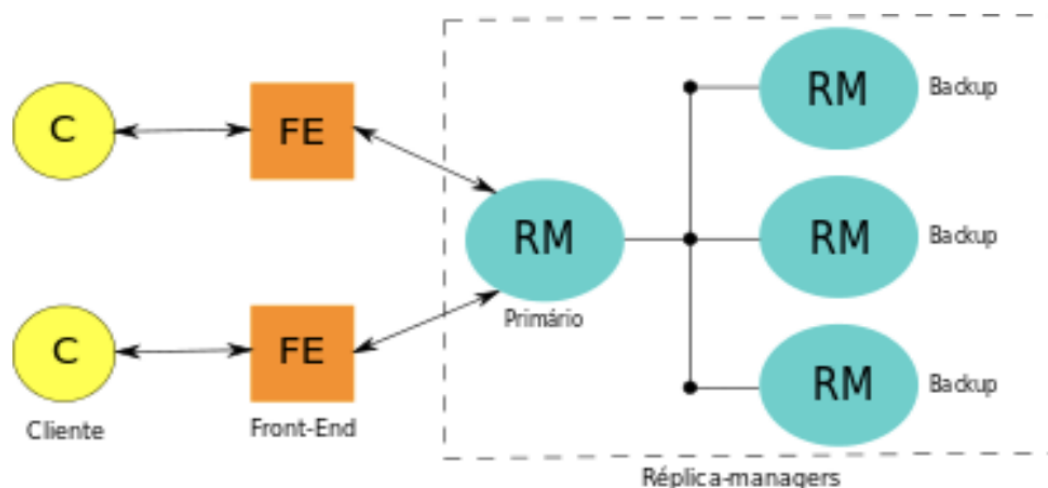


Figura 1. Esquema de Replicação Passiva

4.1. Replicação Passiva

A replicação passiva é um mecanismo fundamental para garantir a disponibilidade e a consistência dos dados em sistemas distribuídos. Nessa abordagem, todas as operações de

escrita, como *uploads*, deleções ou modificações de arquivos são inicialmente processadas por um servidor primário. O cliente, ao realizar uma operação, envia sua requisição ao *frontend*, que atua como intermediário e encaminha a solicitação ao servidor primário responsável. As subseções seguintes descrevem a implementação da replicação passiva em nossa implementação.

4.1.1. Fluxo de Replicação Passiva

O primário aceita conexões de clientes e de múltiplos *backups*, mantendo uma lista dinâmica de conexões (*backupSockets*). Quando um cliente realiza uma operação de escrita (upload ou delete), o primário executa a operação localmente e, em seguida, propaga o comando para todos os *backups* conectados, enviando pacotes estruturados (*packet*) contendo informações do usuário, nome do arquivo e dados relevantes. Os *backups* recebem esses comandos, extraem as informações e aplicam as operações localmente usando métodos do *FileManager*, garantindo que o estado dos arquivos permaneça consistente entre primário e réplicas.

4.1.2. Identificação e Registro de Backups

Ao iniciar, cada *backup* conecta-se ativamente ao primário e envia uma mensagem de identificação (*handshake*). O primário reconhece a conexão como sendo de um *backup* e registra o socket correspondente em sua estrutura de controle (*backupSockets*). Esse mecanismo permite que o primário propague operações para todos os *backups* conectados, sem a necessidade de conhecer previamente seus endereços IP ou portas.

4.1.3. Estruturas de Dados Principais

A replicação utiliza estruturas como *ReplicaConfig* (para configuração de papel e conexões), *backupSockets* (para controle das conexões de *backups*), *FileManager* (para manipulação de arquivos e diretórios dos usuários) e *packet* (para encapsulamento dos comandos e dados transmitidos). *Mutexes* (*pthread_mutex_t*) são empregados para garantir acesso concorrente seguro a recursos compartilhados, como arquivos e listas de conexões.

4.1.4. Gestão Dinâmica de Backups

O primário pode lidar com a chegada e saída dinâmica de *backups*: qualquer *backup* pode conectar-se ao primário a qualquer momento, e o primário automaticamente passa a propagar operações para ele. Se um *backup* desconectar, o primário remove seu socket da lista, mantendo a propagação apenas para os *backups* ativos. Isso permite que o sistema se adapte a mudanças de topologia sem comprometer a consistência dos dados.

5. Problemas encontrados durante o desenvolvimento

5.0.1. Identificação Incorreta de Conexões

Inicialmente, o primário tratava todas as conexões como *backups*, esperando um handshake de identificação mesmo de clientes comuns. Isso causava bloqueios e falhas de protocolo. O problema foi resolvido implementando uma leitura não destrutiva (`MSG_PEEK`) dos primeiros bytes da conexão: apenas conexões que enviam explicitamente o handshake “BACKUP” são registradas como *backups*, enquanto as demais seguem o fluxo normal de clientes.

5.0.2. Propagação de Operações para Backups

O primário tentava abrir uma nova conexão para cada *backup* a cada operação, o que não condiz com o modelo correto de replicação passiva. O código foi ajustado para manter uma lista de sockets de *backups* conectados (`backupSockets`) e propagar comandos de escrita para todos eles, utilizando conexões já estabelecidas.

5.0.3. Formato do Payload e Sincronização de Arquivos

Havia inconsistências no envio das informações de usuário e nome de arquivo para os *backups*, o que impedia a correta criação de diretórios e armazenamento dos arquivos. O problema foi resolvido padronizando o payload dos pacotes de replicação no formato “usuario:arquivo”, permitindo que os *backups* extraíssem corretamente as informações e aplicassem as operações localmente.

5.0.4. Simplificação dos Argumentos de Inicialização

O código original exigia que o primário recebesse os IPs e portas de todos os *backups* por linha de comando, o que não era necessário no modelo correto. O processo foi simplificado para que apenas o *backup* precise saber o IP e porta do primário, tornando a inicialização mais flexível e aderente à arquitetura de replicação passiva.

5.0.5. Consistência de Sequência nas Respostas ao Cliente

Durante a sincronização de diretórios, o servidor enviava notificações com número de sequência (`seqn`) igual a zero, causando erros de protocolo nos clientes. O problema foi resolvido garantindo que todas as respostas relacionadas a um comando utilizem o mesmo número de sequência do comando original, permitindo que os clientes reconheçam corretamente as respostas.

Referências

Garcia-Molina, H. (1982). Elections in a distributed computing system. In *IEEE Transactions on Computers*, volume C-31, pages 48–59.

Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319.